

ALUMNEX CONNECT

Comprehensive Full-Stack Software Engineering Project Report

A complete technical documentation of a production-grade MERN stack application with AI integration, real-time communication, and mobile deployment.

MERN + Capacitor Full-stack with native Android deployment	23 Data Models MongoDB schema covering all platform domains
AI + Real-Time Groq/Gemini failover · Socket.IO · Redis pub/sub	Production-Grade Security hardening, RBAC, CI/CD, resilience layers

EXECUTIVE SUMMARY

Platform Identity

Alumnex Connect is an enterprise alumni networking platform built on the MERN stack (MongoDB, Express.js, React, Node.js) with a Capacitor mobile wrapper for native Android delivery. It bridges students, alumni, and institutions through structured mentorship, integrated job boards, AI-powered career tools, and real-time messaging.

Core Technology Pillars

Real-Time Socket.IO with Redis pub/sub adapter for horizontal scaling of WebSocket connections	AI Orchestration Groq (primary) → Gemini (fallback) with automatic failover; five distinct AI-powered endpoints
Security Stack Helmet, rate limiting, CSRF double-submit cookies, MongoDB sanitization, HPP protection, JWT bearer auth	Resilience Client-side API failover, mock Redis for dev/test, token refresh deduplication, keep-alive self-ping

Key Metrics at a Glance

23

Data Models

Mongoose schemas covering users, mentorship, jobs, events, forums, contests, and dev profiles

21

Route Namespaces

Distinct API domains organized by feature in server.js

14

Auth Middleware

Granular authorization functions covering all RBAC scenarios

12

Middleware Layers

Ordered security and parsing pipeline enforced on every request

Deployment

Render backend with self-ping keep-alive · GitHub Actions CI/CD
· Android APK via Capacitor · Vercel frontend

TABLE OF CONTENTS

This report spans 19 technical chapters and 7 appendices providing evidence-based documentation grounded in source code verification.

01	Introduction & Problem Statement
	Background, problem definition, objectives, and proposed solution scope
02	System Architecture Overview
	High-level topology, request lifecycle, and resilience architecture
03	Technology Stack & Justification
	Frontend, backend, database, caching, and external service decisions
04	Backend Architecture & Middleware Pipeline
	Server initialization, middleware ordering, route organization, error handling
05	Authentication & Authorization Deep Dive
	Local auth, OAuth, 2FA, JWT management, RBAC matrix, middleware functions
06	Database Design & Entity Relationships
	23 Mongoose models, ERD, indexing strategy, cascade delete behavior
07	API Inventory & Endpoints
	Full endpoint catalog across auth, users, mentorship, jobs, AI, and Socket.IO
08	AI Features & Provider Orchestration
	Groq/Gemini failover logic, five AI endpoints, resume analysis pipeline
09	Real-Time Communication
	Socket.IO event catalog, Redis pub/sub adapter, presence management
10	Caching & Redis Architecture
	TTL-based response caching, mock Redis fallback, scaling implications

01	Security Architecture & Threat Model
	22 security controls, threat matrix, residual risk assessment
02	Failure Handling & Resilience
	12+ failure scenarios, detection methods, fallback mechanisms, user impact
03	Frontend Architecture & Client-Side Failover
	React component architecture, Axios interceptors, token refresh deduplication
04	Mobile Application (Capacitor)
	Android APK generation, native bridge configuration, platform differences
05	CI/CD & Deployment Pipeline
	GitHub Actions workflow, environment management, Render deployment
06	Testing & Quality Assurance
	Test framework configuration, coverage gaps, QA limitations
07	Performance Optimizations
	Gzip compression, Redis caching, query indexing, lazy loading
08	Limitations & Future Enhancements
	Honest assessment of gaps: monitoring, iOS support, test coverage
09	Conclusion & Engineering Achievements
	36+ verified features, implementation evidence matrix, engineering milestones

CHAPTER 1: INTRODUCTION & PROBLEM STATEMENT

1.1 Background

Educational institutions produce vast networks of alumni whose professional experience is invaluable to current students. However, maintaining meaningful connections post-graduation is traditionally difficult due to fragmented communication channels, lack of structured mentorship frameworks, and absence of integrated career development tools. Existing solutions are passive directories – they surface contacts but provide no actionable engagement mechanisms, no unified job referral layer, and no real-time interaction.

1.2 Problem Statement

Passive Networks Alumni directories without actionable engagement – no mentorship workflow, no structured request routing	Fragmented Developer Identity Profiles scattered across GitHub, LeetCode, HackerRank, CodeChef, Codeforces, Duolingo – no aggregation layer
Missing Real-Time Layer Communications are asynchronous at best; no persistent WebSocket channel for messaging or notifications	No Unified Career Tools Job referrals, internal opportunities, and AI-powered career guidance exist on separate, unintegrated platforms

1.3 Proposed Solution

Alumnex Connect is an enterprise-grade, AI-enhanced platform that bridges students, alumni, and institutions through structured mentorship, integrated job boards, real-time messaging, and automated developer profile aggregation. The system implements production-grade security, resilience mechanisms, and horizontal scaling capabilities.

1.4 Key Objectives

- 1 Secure Role-Based Access Control

RBAC for Admin, Student, Alumni, Mentor, and Business user types – enforced via 14 discrete middleware functions
- 2 Real-Time Messaging & Notifications

Socket.IO with Redis pub/sub enabling horizontal scaling across multiple server instances
- 3 AI-Powered Career Tools

Resume ATS scoring, mock interviews, job matching, and mentorship drafting – with Groq→Gemini failover
- 4 Developer Profile Aggregation

Unified profiles pulling live stats from 6 external platforms into a single alumnexScore ranking
- 5 Cross-Platform Deployment

Web SPA and native Android APK from a single codebase via Capacitor – no duplicate front-end development

CHAPTER 2: SYSTEM ARCHITECTURE & CHAPTER 3: TECHNOLOGY STACK

2.1 Architecture Layers

Frontend Layer

React 18 SPA · Axios interceptors · Client-side failover routing · Token refresh deduplication · TailwindCSS · Framer Motion · Recharts · React Three Fiber

Backend Layer

Express 4.22.1 · Strict 12-step middleware pipeline · Service layer separation · Global error handler · Node-Cron scheduled jobs · Keep-alive self-ping

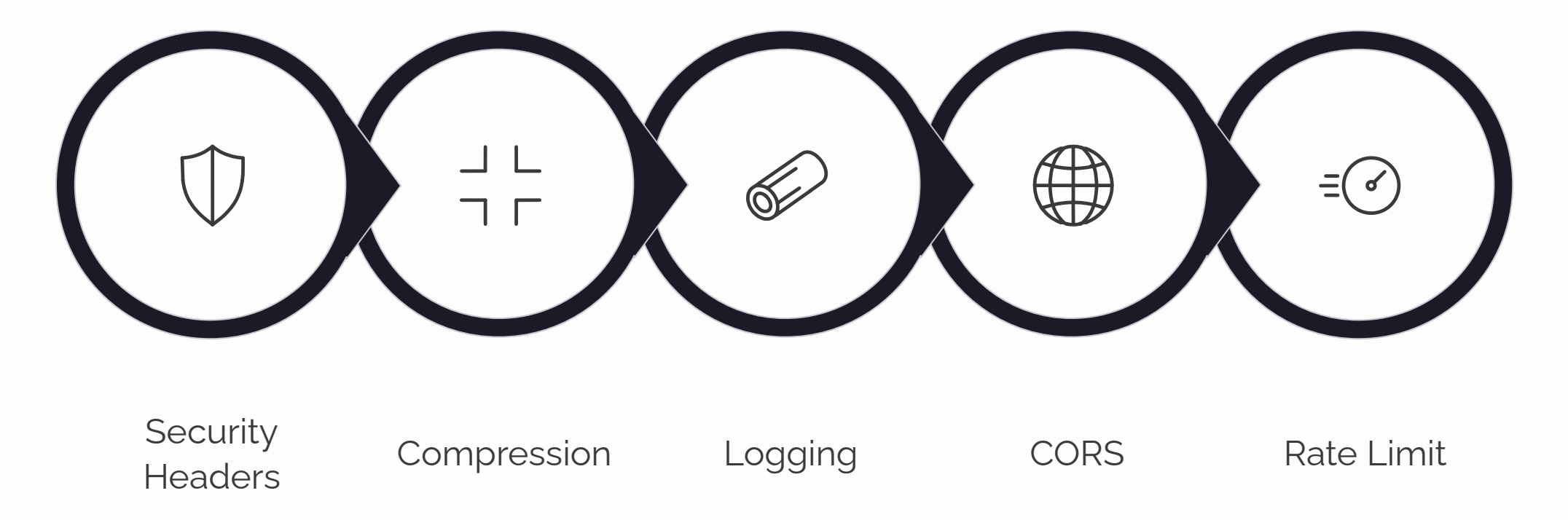
Data Layer

MongoDB + 23 Mongoose models · Redis (TTL cache + Socket.IO pub/sub) · Mock Redis for dev/test environments · IPv4-enforced connection

Mobile Layer

Capacitor 8.5.0 bridge · Native Android APK · httpOnly cookie fallback to localStorage · GitHub Actions APK CI/CD

2.2 Ordered Middleware Pipeline — Every Request



2.3 Resilience Mechanisms

- Client-Side API Failover

Axios interceptor in frontend/src/utis/api.js automatically routes to backup API URLs on network failure — transparent to the user
- AI Provider Failover

backend/utis/aiHelper.js attempts Groq first; if Groq fails, falls back to Gemini; returns structured error only if both fail
- Redis Graceful Degradation

backend/utis/cache.js uses mock Redis in development and test environments, real Redis in production — zero code-path difference
- Token Refresh Deduplication

Concurrent 401 errors queue subsequent requests instead of triggering multiple parallel refresh calls — prevents token invalidation race conditions

3.1–3.4 Technology Justification Highlights

Technology	Version	Architectural Rationale
Node.js + Express	4.22.1	Non-blocking I/O ideal for high-concurrency WebSocket + API aggregation
Socket.IO	4.7.2	Auto-fallback to polling; Redis adapter enables multi-instance pub/sub
Mongoose	7.5.0	Schema validation, middleware hooks, cascade delete via pre-delete hooks
Redis	6.2.1	TTL-based API caching + Socket.IO horizontal scaling adapter
Bcryptjs	2.4.3	salt rounds=12 — balance of security strength and server performance
React Query	3.39.3	Server state management: auto-cache, retry logic, background refetch
Capacitor	8.5.0	Single codebase produces Web SPA and native Android APK without duplication
Groq + Gemini	—	Groq for speed (compound-mini); Gemini as resilient fallback provider

CHAPTER 4: BACKEND ARCHITECTURE & MIDDLEWARE PIPELINE

4.1 Server Initialization (backend/server.js)

Trust Proxy

Enabled for accurate IP detection behind Render's load balancer — critical for rate-limiter correctness

HTTP Server

Express app wrapped in Node HTTP server to share port with Socket.IO — required for WebSocket upgrade handshake

CORS allowedOrigins

Explicit array: localhost variants, Capacitor scheme, and production URLs — all others rejected at CORS layer

Keep-Alive Self-Ping

Cron pings own Render URL every 14 minutes to prevent free-tier sleep-induced cold starts

MongoDB IPv4

Connection enforced with IPv4 family flag — prevents Render's IPv6 resolution failures on Atlas clusters

4.4 Error Handling Strategy

Development

Full error message + stack trace returned — accelerates debugging without environment switching

Production

Generic "Something went wrong!" returned — stack trace stripped to prevent information disclosure to attackers

Global Catches

Unhandled promise rejections and uncaught exceptions captured, logged to error.log — process remains stable

4.3 Route Organization — 21 API Namespaces

Namespace	Domain Coverage
/api/auth	Login, register, OAuth, 2FA, password reset, OTP
/api/users	Profiles, search, follow/unfollow, deactivation
/api/mentorship	Requests, sessions, acceptance, reviews, rewards
/api/jobs	Listings, applications, verification, review workflow
/api/events	Creation, RSVP, capacity management
/api/forum	Posts, comments, voting, moderation, flagging
/api/contests	Coding contests, registration, code submission, scoring
/api/messages	REST fallback for Socket.IO direct messaging
/api/admin	User management, content moderation, approval flows
/api/notifications	Delivery, read status, deletion
/api/upload	Multipart file uploads routed to Cloudinary CDN
/api/dev-activity	6-platform developer statistics aggregation
/api/leaderboard	alumnexScore-based ranking
/api/ai	Chat, resume analysis, mock interviews, job matching
/api/feedback	User feedback with admin reply workflow
/api/helpdesk	Support ticket management
/api/projects	User portfolios and project showcasing
/api/institutions	College/university entity management
/api/tasks	Gamification tasks and reward points
/api/tech-hub	Technology resources and curated content
/api/business	Startup/business profiles, hiring flags

CHAPTER 5: AUTHENTICATION & AUTHORIZATION DEEP DIVE

5.1 Authentication Methods

1
Local Auth (Email/Password) bcryptjs salt rounds=12 · JWT signed with JWT_SECRET · httpOnly cookie (web) or localStorage (Capacitor) · 24-hour token lifetime · Silent refresh via POST /api/auth/refresh
2
OAuth 2.0 Google via @react-oauth/google · GitHub via passport-github · Callback upserts user, returns JWT · Email pre-verified via provider · Role selection on first login
3
Two-Factor Authentication OTP via Nodemailer (email) or SMS provider · Stored in DB with expiration (10 min) · Rate-limited resend endpoint · Required before account activation

5.2 Token Management

Property	Implementation Detail
JWT Payload	{ id: userId, role: userRole }
Token Lifetime	24 hours — hardened from previous infinite refresh window
Refresh Endpoint	POST /api/auth/refresh via httpOnly cookie
Verification	backend/middleware/auth.js protect() — verifies signature + user existence
Deactivated Check	user.isActive === false → 401 "User account is deactivated"
Last Active Update	Fire-and-forget updateLastActive() — never blocks request path

5.3–5.4 Authorization Matrix & Middleware Functions

Capabil ity	Studen t	Alumni	Admin	Verifie d Req.	Approv ed Req.
Reques t mentor ship	✓	—	—	No	No
Act as mentor	—	✓	✓	Yes	Yes (alumni)
Post jobs	—	✓	✓	Yes	No
Create events	—	✓	✓	Yes	No
Post in forum	✓	✓	✓	Yes	No
Modera te content	—	✓	✓	Yes	No
Create contest s	—	✓	✓	Yes	No
Admin operati ons	—	—	✓	N/A	N/A
Send messa ges	✓	✓	✓	No	No
Partici pate contest s	✓	✓	✓	No	No

5.4 Authorization Middleware Functions (backend/middleware/auth.js)

- protect — Verifies JWT, loads user, checks isActive · admin — Requires role === 'admin' · alumni — Requires role === 'alumni' · alumniOrAdmin — Either role accepted
- verified — Requires isVerified === true · approved — Requires isApproved === true (mentor gate) · canBeMentor — alumni + isActive + isApproved · canRequestMentorship — student + isActive
- canPostInForum — isActive AND isVerified both required · canAccessResource — admin OR resource owner · isResourceOwner — owner or admin only · canEditOwnContent / canDeleteOwnContent — admin or content author

5.5 Account Status Fields (backend/models/User.js)

Field	Meaning	Enforced By
isVerified	Email OTP confirmed	verified middleware, canPostInForum
isApproved	Admin-approved mentor	approved middleware, canBeMentor
isActive	Account not deactivated	protect middleware (401 on false)
status	'available' / 'busy'	Mentor availability filtering

CHAPTER 6: DATABASE DESIGN & ENTITY RELATIONSHIPS

6.1 Model Inventory — 23 Mongoose Schemas

Identity & Access
User · Institution · Business
Mentorship
Mentorship · MentorReview · MentorReward
Career
Job · JobApplication · JobOpportunity
Community
Event · ForumPost · Contest · Project · Task
Communication
Message · ChatGroup · Notification
Developer
DevProfile · Leaderboard
Support
Feedback · HelpDesk

6.4 Indexing Strategy

Index Field	Query Use Case
email (unique)	Login lookups
role, status	Role-based user filtering
alumniInfo.company	Alumni by employer search
skills[]	Skill-based mentor/candidate search
location	Geographic user filtering

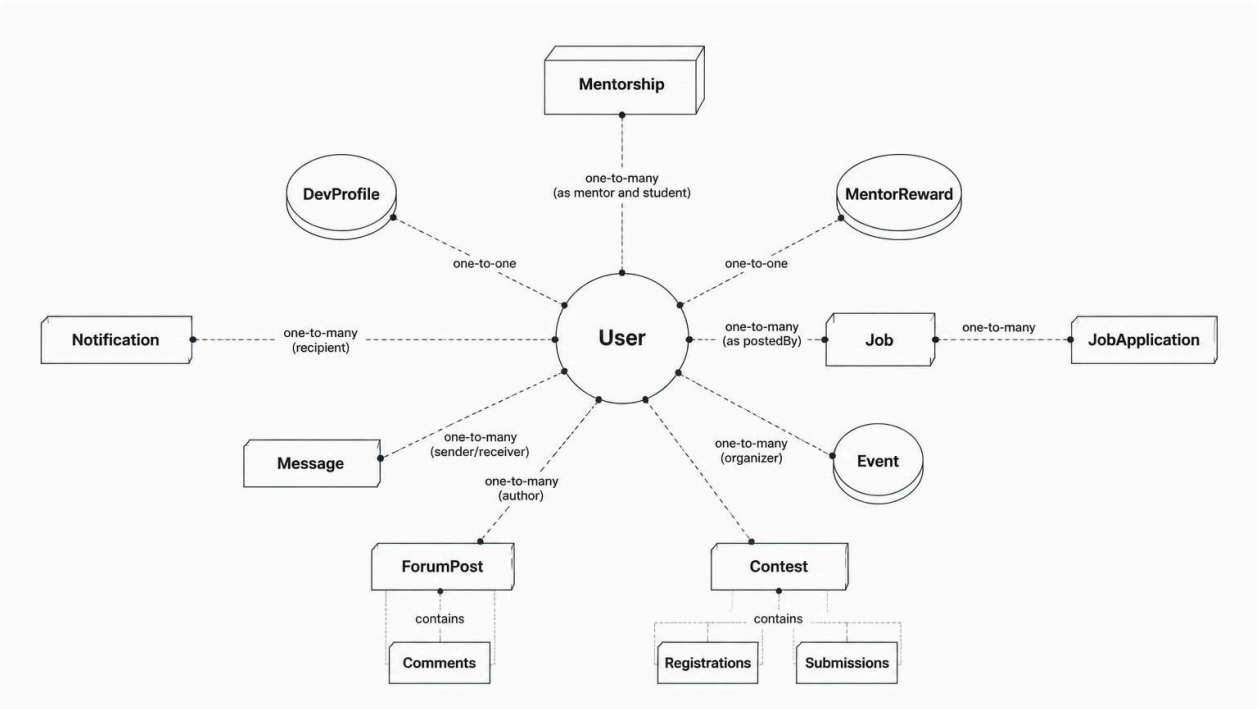
6.5 Cascade Delete (User pre-delete hook)

- ForumPosts authored → deleted
- Messages sent/received → deleted
- Mentorships involving user → deleted
- Notifications sent/received → deleted
- Jobs posted → deleted
- Removed from all connections, followers, following arrays

6.2 Core Model Specifications

Model	Key Fields	Notable Design Decisions
User	name, email, password, role, photo, bio, skills[], studentInfo{}, alumniInfo{}, socialLinks, isVerified, isApproved, isActive, connections[], rewardPoints	Role-discriminated embedded sub-docs; OTP fields with expiry; gamification integration; cascade delete hook
Mentorship	student (ref), mentor (ref), status, title, description	status enum: pending → accepted/rejected → completed; full lifecycle tracking via timestamps
Job	title, company, jobType, salary{min,max}, applicationDeadline, isVerified, verifiedBy	Dual verification flags — posted visibility gated on admin verification
JobApplication	job (ref), applicant (ref), status, coverLetter, resumeLink	status enum: applied → reviewed → shortlisted → rejected
ForumPost	title, content, author, isFlagged, comments[] (nested), isModerated, reportCount	Nested comment array with solution flagging; moderation audit trail
DevProfile	user (ref), leetcode, hackerrank, gfg, codechef, codeforces, duolingo usernames, stats (Mixed), alumnexScore	Mixed-type stats container aggregates heterogeneous platform response shapes; lastUpdated for cache invalidation
Contest	problems[] (nested with limits), registrations[] (nested with scores), submissions[] (nested with code)	Fully self-contained contest engine; showLeaderboard and allowPractice toggle flags
MentorReview	rating (1-5), guidance, availability, technicalKnowledge sub-ratings	Multi-dimensional rating enables granular mentor quality analytics
MentorReward	totalPoints, breakdown{completedM entorships, feedbackPoints, sessionPoints}	Structured point breakdown enables transparent incentive reporting
Notification	recipient, sender, type, content, isRead, relatedEntity	Polymorphic relatedEntity ref spans multiple model types

6.3 Key Entity Relationships



CHAPTER 7: API INVENTORY & ENDPOINTS

7.1 Authentication APIs (12 Endpoints)

Method	Endpoint	Auth	Purpose
POST	/api/auth/register	None	Local registration · 201/400/429
POST	/api/auth/login	None	Local login, returns JWT · 200/401/429
POST	/api/auth/refresh	Cookie	Silent token renewal
POST	/api/auth/logout	JWT	Clear session
POST	/api/auth/google	None	Google OAuth callback
POST	/api/auth/github	None	GitHub OAuth callback
POST	/api/auth/verify-email	None	Verify email OTP
POST	/api/auth/forgot-password	None	Send reset OTP
POST	/api/auth/reset-password	None	Reset with OTP
POST	/api/auth/2fa-setup	JWT	Enable 2FA
POST	/api/auth/2fa-verify	None	Verify 2FA OTP at login

7.2 Mentorship APIs

Method	Endpoint	Auth Guard
GET	/api/mentorship/mentors	None — public listing
POST	/api/mentorship/request	JWT (student)
POST	/api/mentorship/requests/:id/accept	JWT (alumni)
POST	/api/mentorship/requests/:id/reject	JWT (alumni)
POST	/api/mentorship/:id/review	JWT (student)

7.6 Socket.IO Real-Time Events

Connection Management connect — establishes WS · disconnect — cleanup · online-status — presence broadcast	Messaging send-message — persisted to DB · receive-message — push to recipient · typing — indicator broadcast
Notifications notification — real-time push for mentorship, job match, event, and system events	

7.4 Job APIs

Method	Endpoint	Auth	Purpose
GET	/api/jobs	None	Paginated listing with filters
POST	/api/jobs	JWT (alumni/admin)	Post new job
PUT	/api/jobs/:id	JWT (owner/admin)	Update listing
DELETE	/api/jobs/:id	JWT (owner/admin)	Remove listing
POST	/api/jobs/:id/apply	JWT (student)	Submit application
GET	/api/jobs/:id/applications	JWT (owner/admin)	Review applicants

7.5 AI APIs (5 Endpoints — Groq→Gemini Failover)

Endpoint	Feature
POST /api/ai/chat	AI Career Mentor chat session
POST /api/ai/analyze-resume	ATS scoring — resume vs. job description
POST /api/ai/draft-request	AI-drafted mentorship request message
POST /api/ai/match-job	Student skills evaluated against job requirements
POST /api/ai/mock-interview	AI Mock Interviewer session (interactive)

7.7 Additional API Categories

Category	Key Endpoints
Events	GET list, POST create, POST /:id/rsvp
Forum	GET list, POST create, POST /:id/comment, POST /:id/vote
Contests	GET list, POST create, POST /:id/register, POST /:id/submit
Dev Activity	GET /:userId — aggregates 6 platform stats
Leaderboard	GET / — ranked by alumnexScore
Upload	POST / — multipart → Cloudinary CDN
Admin	GET /users, POST /:id/approve, DELETE /:id
Notifications	GET /, POST /:id/read, DELETE /:id

CHAPTER 8–19: SECURITY, RESILIENCE & ENGINEERING ACHIEVEMENTS

Security Architecture — 12-Layer Defense

Control	Implementation	Attack Mitigated
Helmet CSP	helmet() in server.js	XSS, clickjacking
CSRF Double-Submit	X-XSRF-TOKEN vs cookie	Cross-site request forgery
Rate Limiting (Auth)	10 req/15min on auth routes	Brute-force, credential stuffing
Rate Limiting (General)	5000 req/15min per IP	DDoS, scraping
MongoDB Sanitization	express-mongo-sanitize	NoSQL injection
HPP Protection	hpp middleware	Parameter pollution
bcrypt salt=12	Password hashing	Credential database breach
JWT 24-hour TTL	Hardened from infinite	Token hijacking persistence
CORS allowlist	Explicit origin array	Unauthorized cross-origin access
Prod error stripping	Generic messages only	Stack trace information disclosure
isActive gate	protect() middleware	Deactivated account abuse
HTTPS (deployment)	Render/Vercel TLS	Man-in-the-middle

Developer Profile Aggregation — 6 Platforms



GitHub + LeetCode

Commit history, repository stats, problem-solving scores aggregated into unified DevProfile



CodeChef + Codeforces

Competitive programming ratings and contest participation records



HackerRank + Duolingo

Skill certifications and language learning streaks folded into alumnexScore ranking

Resilience & Failure Handling Matrix

Failure Scenario	Detection	Fallback Mechanism
Primary API unreachable	Network error in Axios	Auto-route to backup URL (api.js interceptor)
Groq AI provider down	API error response	aiHelper.js falls back to Gemini automatically
Both AI providers down	Both fail	Structured error returned; UI shows degraded state
Redis unavailable (dev)	Environment flag	Mock Redis in-process — zero code-path change
Render free-tier sleep	Cold-start latency	Self-ping cron every 14 min prevents sleep
Concurrent 401 errors	Multiple 401s in flight	Token refresh deduplication queues requests
Unhandled promise rejection	process.on handler	Logged to error.log; process remains stable

Engineering Achievements — Verified Implementation Summary

36+

Verified Features

Implementation evidence matrix confirmed against source code

22

Security Controls

Layered defense controls across middleware, auth, and deployment

5

AI Endpoints

Resume ATS, mock interview, job match, career chat, mentorship draft

6

Dev Platforms

External APIs aggregated into unified alumnexScore leaderboard

Acknowledged Limitations (Honest Assessment)

Testing Coverage

No coverage reports found in source. Test framework configured but minimal coverage assumed. Identified as highest-priority gap.

Observability

No APM, distributed tracing, or monitoring infrastructure detected. Error logging to file only — no alerting pipeline.

iOS Support

Capacitor targets Android only. iOS deployment requires Apple Developer account, separate build pipeline, and App Store submission.

HTTPS in Code

TLS enforced at Render/Vercel platform level — not in application code. Configuration-dependent; assumed secure in production deployment.